



Creación de un chatbot con IA capaz de buscar información en fuentes externas

Grado en Ingeniería Informática

Trabajo Fin de Grado

Autor:

Mario Martínez Turpín

Tutor/es:

Jose Antonio Miñarro Gimenez

Catalina Martínez Costa



**Facultad
Informática
Universidad
Murcia**

26 de Mayo de 2026

Creación de un chatbot con IA capaz de buscar información en fuentes externas

MMT Chat

Autor

Mario Martínez Turpín

Tutor/es

Jose Antonio Miñarro Gimenez

Departamento de Informática y Sistemas

Catalina Martínez Costa

 **Facultad de
Informática**

Grado en Ingeniería Informática

CIIC

UNIVERSIDAD DE
MURCIA



Murcia, 26 de Mayo de 2026

Agradecimientos

Me gustaría expresar mi agradecimiento a mis tutores, Jose Antonio Miñarro Gime-
nez y Catalina Martínez Costa, por su guía a lo largo de todo el proceso. Su ayuda ha
sido fundamental para navegar los procedimientos formales y darle forma al trabajo.

Asimismo, quiero agradecer a mi familia por el apoyo incondicional y la confianza
que han depositado en mí durante todo este tiempo. Sin ellos, este proyecto no habría
sido posible.

También quiero destacar a mi pareja, Natalia, por su amor, paciencia y comprensión,
que tanto me han ayudado a seguir adelante durante estos meses.

Finalmente, dedico este trabajo a todas las personas que han contribuido a que
esta experiencia sea enriquecedora y exitosa. Su ayuda y confianza me han permitido
alcanzar mis objetivos y convertir esta tesis en una realidad.

Declaración firmada sobre originalidad del trabajo

D./Dña. **Mario Martínez Turpín**, con DNI **49183039T**, estudiante de la titulación de **Grado en Ingeniería Informática** de la Universidad de Murcia y autor del TF titulado **“Creación de un chatbot con IA capaz de buscar información en fuentes externas”**.

De acuerdo con el Reglamento por el que se regulan los Trabajos Fin de Grado y de Fin de Máster en la Universidad de Murcia (aprobado C. de Gob. 30-04-2015, modificado 22-04-2016 y 28-09-2018), así como la normativa interna para la oferta, asignación, elaboración y defensa de los Trabajos Fin de Grado y Fin de Máster de las titulaciones impartidas en la Facultad de Informática de la Universidad de Murcia (aprobada en Junta de Facultad 27-11-2015)

DECLARO:

Que el Trabajo Fin de Grado presentado para su evaluación es original y de elaboración personal. Todas las fuentes utilizadas han sido debidamente citadas. Así mismo, declara que no incumple ningún contrato de confidencialidad, ni viola ningún derecho de propiedad intelectual e industrial

Murcia, a 26 de Mayo de 2026



Fdo.: Mario Martínez Turpín
Autor del TF

Resumen

En la era digital actual, el acceso eficiente y preciso a grandes volúmenes de información estructurada y no estructurada representa uno de los mayores desafíos en el ámbito de las Ciencias de la Computación. Históricamente, la Web Semántica y los Grafos de Conocimiento (*Knowledge Graphs*) han proporcionado un marco tecnológico robusto para modelar información de manera determinista mediante tripletas (Sujeto, Predicado, Objeto) basadas en el estándar RDF (Resource Description Framework). El acceso a esta información altamente estructurada y veraz se realiza tradicionalmente a través de SPARQL, un lenguaje de consulta potente pero con una curva de aprendizaje pronunciada que supone una barrera técnica insalvable para la mayoría de los usuarios no expertos.

De forma paralela, el reciente auge de los Modelos de Lenguaje de Gran Escala (LLMs) ha revolucionado la interacción humano-máquina. Estos modelos permiten establecer interfaces conversacionales fluidas en lenguaje natural, facilitando enormemente el acceso a la información. Sin embargo, su naturaleza estocástica los hace vulnerables a problemas inherentes como las denominadas "alucinaciones" (la generación de respuestas que resultan gramaticalmente correctas y plausibles, pero que son total o parcialmente falsas) y presentan serias dificultades para recuperar datos precisos y actualizados de dominios de conocimiento altamente específicos.

El presente Trabajo de Fin de Grado (TFG) aborda la convergencia de ambas tecnologías con el objetivo de aunar la flexibilidad y capacidad de comprensión del lenguaje natural propia de los LLMs con la precisión factual inmutable de un Grafo de Conocimiento. Para lograr este ambicioso objetivo, se ha diseñado, implementado y evaluado un agente conversacional inteligente (chatbot) especializado en el dominio cinematográfico. A diferencia de un chatbot tradicional que responde basándose puramente en los patrones aprendidos en su entrenamiento, este sistema actúa como una interfaz transparente que traduce la intención del usuario a consultas SPARQL válidas, ejecutándolas contra una base de datos local y devolviendo la respuesta exacta.

La arquitectura del sistema propuesto se fundamenta en una evolución del paradigma de Generación Aumentada por Recuperación (RAG), elevándolo hacia un sistema de Inteligencia Artificial Agéntica utilizando el *framework* LangChain. La solución tecnológica se ha estructurado en tres capas principales. En la capa de presentación, se ha desarrollado una interfaz gráfica de usuario amigable mediante Streamlit, que no solo facilita la interacción conversacional, sino que también expone de manera transparente el razonamiento interno del agente ("Chain-of-Thought") y las consultas generadas en tiempo real. En la capa lógica, el núcleo cognitivo del sistema, impulsado por el

modelo Llama3 ejecutado localmente mediante Ollama, se encarga de la extracción de entidades, su desambiguación contra el vocabulario de Wikidata y la generación dinámica de código SPARQL. Finalmente, la capa de datos descansa sobre un Grafo de Conocimiento local gestionado mediante la librería RDFLib en Python.

Un hito técnico fundamental de este trabajo ha sido la construcción de este repositorio de datos semántico. Depender directamente de *endpoints* públicos como el de Wikidata durante la ejecución en tiempo real introducía niveles inaceptables de latencia y riesgo de bloqueos por límites de peticiones (APIs *rate limits*). Por ello, se implementó un proceso de extracción *offline* estructurado para aislar un subgrafo densamente conectado del dominio cinematográfico (abarcando miles de películas, directores, actores, géneros y fechas de estreno), el cual fue serializado en formato *Turtle* (.ttl). Esto proporciona al agente un entorno de ejecución local, excepcionalmente rápido y completamente determinista.

El aspecto más innovador de la implementación radica en el mecanismo autónomo de autocorrección del agente, denominado "bucle de reflexión" (*Reflexion Loop*). La traducción de lenguaje natural a sintaxis SPARQL estricta es propensa a errores. Para mitigarlo, el agente cuenta con un diccionario de propiedades permitidas altamente restrictivo (por ejemplo, `wdt:P57` exclusivamente para referenciar a un "director") que acota drásticamente su espacio de búsqueda. Si el modelo genera una consulta con errores de sintaxis o que no compila, la capa de datos captura la excepción (*traceback*) de RDFLib y, en lugar de fallar de manera abrupta, realimenta al LLM con este error de compilación inyectado en un *prompt* de reflexión. De este modo, el agente analiza lógicamente su propia equivocación y reescribe la consulta desde cero, logrando un notable incremento en la robustez, resiliencia y autonomía del sistema.

Para validar el sistema propuesto de forma objetiva, rigurosa y cuantitativa, se desarrolló ad-hoc un marco de evaluación automatizado. Se confeccionó un conjunto de pruebas (*dataset*) compuesto por 61 consultas complejas en lenguaje natural que abarcaban diversas relaciones semánticas de dificultad incremental, y se llevó a cabo un estudio de rendimiento comparativo entre distintos modelos de lenguaje de gran escala. El modelo seleccionado para la versión final del proyecto, Llama3, demostró un rendimiento excepcional: alcanzó una tasa de precisión del 90.16% en la fase crítica de extracción y desambiguación de entidades, e idéntico 90.16% de éxito global en la generación de código SPARQL válido y ejecutable que retornó la respuesta correcta a la pregunta planteada.

Además de las tasas de éxito en precisión, se midió exhaustivamente el tiempo de ejecución por pregunta, un factor sumamente crítico para asegurar una buena experiencia de usuario en cualquier asistente conversacional interactivo. Llama3 demostró un rendimiento óptimo en este aspecto, promediando un tiempo de ejecución de respuesta de tan solo 3.66 segundos por pregunta, erigiéndose de manera destacada como el modelo más equilibrado y rápido de entre todos los evaluados en este trabajo. Un análisis pormenorizado de los escasos fallos residuales reveló que estos se debían, en su inmensa mayoría, a ambigüedades idiomáticas extremas por parte del usuario o a

cuellos de botella aislados en las llamadas de desambiguación de entidades, y no a fallos estructurales o lógicos en la formulación de consultas deductivas por parte del LLM.

En conclusión, los resultados empíricos obtenidos a lo largo de las pruebas confirman sin lugar a dudas que la integración de agentes de LangChain fuertemente tipados con Grafos de Conocimiento locales constituye una solución tecnológica sumamente viable y efectiva para la creación de sistemas de pregunta-respuesta (QA) totalmente transparentes y estructuralmente libres de alucinaciones. El sistema diseñado ha logrado el hito de ocultar la enorme complejidad inherente al modelo de datos de la Web Semántica de cara al usuario final, manteniendo no obstante intacta la rigurosa precisión y veracidad de sus datos subyacentes. El presente trabajo sienta, por tanto, unas bases sólidas y prometedoras para futuras líneas de investigación y desarrollo, tales como la mejora de los algoritmos basados en similitud de vectores (*embeddings*) para lograr una desambiguación semántica libre de fallos, o la ampliación escalonada de la ontología base para llegar a soportar razonamientos deductivos mucho más complejos de múltiples saltos lógicos (*multi-hop reasoning*) que permitan extraer conclusiones de datos aparentemente inconexos.

Palabras clave: Web Semántica, Grafos de Conocimiento, SPARQL, Modelos de Lenguaje (LLMs), Llama3, Generación Aumentada por Recuperación (RAG), LangChain, Inteligencia Artificial Agéntica, Streamlit, Mitigación de Alucinaciones.

Extended Abstract

This Final Degree Project presents the comprehensive design, implementation, and rigorous evaluation of an intelligent conversational agent specialized in the cinematographic domain. The primary objective of this research is to bridge the significant technical gap between the natural language comprehension capabilities of modern Large Language Models (LLMs) and the factual, deterministic precision inherent to Semantic Web Knowledge Graphs. By synergizing these two distinct technological paradigms, the project directly addresses one of the most critical and widely discussed challenges in modern generative Artificial Intelligence: the mitigation and elimination of “hallucinations”—the generation of plausible yet entirely fabricated information. Through the development of an autonomous, self-correcting agent that translates natural language into strict SPARQL queries, this work demonstrates how semantic data can be made accessible to non-expert users while maintaining absolute data veracity and system transparency.

1. Introduction and Motivation

In recent years, the exponential advancement of Large Language Models has fundamentally revolutionized the way users and developers interact with information systems. These models have enabled highly fluent, context-aware conversational interfaces that can generate human-like text across a vast array of topics. However, their underlying stochastic nature—predicting the next most probable token based on statistical weights—makes them inherently unreliable for querying specific, factual, or highly specialized data. They suffer from a static knowledge cutoff, meaning they are unaware of events occurring after their training phase, and most critically, they are prone to hallucinations, confidently asserting incorrect facts when their internal weights lack the precise information requested.

Conversely, the Semantic Web and Knowledge Graphs provide impeccably structured, verifiable, and deterministic data environments. Using the Resource Description Framework (RDF), knowledge is modeled in explicit subject-predicate-object triples, accessible via precise, standardized query languages like SPARQL. Despite its power, SPARQL possesses a remarkably steep learning curve and unforgiving syntax requirements, effectively creating a barrier that prevents non-expert users from accessing this immense wealth of information. Extracting specific cinematic data from a graph containing millions of interconnected nodes requires a deep understanding of the underlying

ontology.

This project proposes a robust hybrid solution to resolve this dichotomy: an autonomous conversational agent that acts as a transparent natural language interface for a local Knowledge Graph. To achieve this, four specific objectives were defined and successfully met:

1. **Precision in SPARQL Generation:** Developing an autonomous agent capable of accurately translating natural language into valid SPARQL, incorporating a self-correction mechanism to recover from syntactic errors.
2. **Advanced RDF Data Management:** Extracting, processing, and locally storing a massive subset of Wikidata in N-Triples format to optimize query performance and ensure data privacy.
3. **LLM Control and Restriction:** Designing a highly structured prompting framework combined with strict property mapping to mathematically limit the LLM’s search space, thereby preventing semantic hallucinations.
4. **Quantitative Evaluation:** Implementing a fully automated evaluation pipeline to objectively measure the system’s accuracy, latency, and resilience across multiple state-of-the-art open-weights models.

2. State of the Art and Theoretical Foundations

The theoretical foundation of this work rests firmly on three interconnected technological pillars: the Semantic Web, Large Language Models, and the evolution of Retrieval-Augmented Generation (RAG) paradigms toward Agentic AI.

The Semantic Web and Knowledge Graphs: Originally envisioned by Tim Berners-Lee, the Semantic Web aims to make internet data machine-readable. At its core is the Resource Description Framework (RDF), which standardizes data representation as directed, labeled graphs. Each piece of information is a triple consisting of a subject (an entity), a predicate (a relationship), and an object (another entity or a literal value). This structure is exceptionally efficient for multi-hop queries where relationships traverse several nodes. Wikidata, a massive, multilingual, and collaboratively edited knowledge base, serves as the primary and authoritative data source for this project due to its unparalleled volume and open semantic structure.

SPARQL: The SPARQL Protocol and RDF Query Language is the W3C standard for querying RDF graphs. While incredibly expressive, allowing for complex filtering, aggregation, and graph traversal, its strict syntactic demands make it entirely inaccessible to the average user. This underscores the necessity for Text-to-SPARQL translation systems.

Large Language Models and their Limitations: Built upon the Transformer architecture, modern LLMs exhibit remarkable linguistic capabilities. Yet, when deployed as standalone knowledge retrieval systems, they fail. Their knowledge is implicitly encoded in billions of parameters, making it impossible to guarantee factual accuracy or trace the provenance of a given statement, ultimately leading to hallucinations.

GraphRAG and Agentic AI: To overcome these limitations, Retrieval-Augmented Generation (RAG) dynamically retrieves external data to ground the LLM's response. This project pushes the traditional vector-based RAG paradigm significantly further by implementing *GraphRAG* or *Text-to-SPARQL RAG*. Instead of retrieving fuzzy text chunks based on semantic similarity, the LLM is tasked with generating deterministic logical code to interrogate a graph. Furthermore, by utilizing the LangChain framework, the system transcends linear execution. It embraces an *Agentic AI* workflow where the LLM functions as a cognitive engine capable of observing outputs, reasoning about errors, planning new strategies, and iteratively refining its approach without human intervention.

3. System Architecture and Methodology

Given the highly experimental, empirical, and non-deterministic nature of advanced prompt engineering and LLM behavior, a rigid waterfall development approach was discarded. Instead, an agile, iterative, and test-driven methodology was adopted throughout the development lifecycle. This involved rapid cycles of data extraction, prompt prototyping, logic refinement, and continuous automated evaluation.

The overall system architecture is modularly divided into three highly cohesive and decoupled main layers, ensuring scalability and maintainability:

- **Presentation Layer (Streamlit):** A highly interactive, responsive, and user-friendly graphical interface built entirely with the Streamlit framework. Beyond serving as a simple chat interface, it incorporates crucial Explainable AI (XAI) features. It provides a seamless experience while simultaneously exposing the internal "Chain-of-Thought" reasoning process to the user. The interface dynamically displays the extracted entities, the generated SPARQL code, and any active error-correction loops, ensuring maximum transparency and user trust.
 - **Logic and Agent Layer (Python & Ollama):** The cognitive core of the application, encapsulated primarily within the `chatLocal.py` module. This layer manages the complex orchestration of natural language understanding, context retention, entity disambiguation, and the core iterative SPARQL generation loop. A critical design decision was the utilization of Ollama as the inference engine. This allows state-of-the-art models (like Llama 3, Mistral, and Gemma 2) to run 100% locally on the host machine. This architecture guarantees absolute data
-

privacy, eliminates reliance on expensive, rate-limited external cloud APIs, and ensures consistent execution environments.

- **Data Layer (Blazegraph & SPARQLWrapper):** Rather than relying on simple in-memory libraries, the system utilizes Blazegraph, a high-performance, enterprise-grade RDF graph database. Blazegraph acts as a local SPARQL endpoint hosting the extracted N-Triples dataset. The Python backend communicates with this endpoint using the SPARQLWrapper library, allowing for rapid query execution and seamless integration.

4. Local Knowledge Graph Construction and Ontology Mapping

A critical architectural decision was to completely avoid querying public SPARQL endpoints (such as the official Wikidata Query Service) during real-time user interactions. Relying on public infrastructure introduces unacceptable latency, unpredictable timeouts, and severe risks of being blocked by strict API rate limits, fundamentally compromising the user experience.

Therefore, a custom, highly tailored data extraction pipeline was developed. The `extraer_dump_cine.py` script was engineered to programmatically query Wikidata entirely offline, extracting a densely connected, self-contained subgraph focusing exclusively on the cinematographic domain. This massive extraction isolates movies, their directors, actors, genres, and release dates, serializing the resulting data into a clean *N-Triples* (.nt) file format. This local repository provides a lightning-fast, secure, and fully controlled environment for the agent to execute queries.

To guarantee that the LLM generates syntactically valid queries that actually match the database schema, a strict ontology mapping strategy was implemented. A configuration file, `mapeo_propiedades.json`, explicitly defines the only allowed semantic properties the model can use. Twelve core properties were meticulously selected for movies (e.g., `wdt:P57` for Director, `wdt:P161` for Cast Member, `wdt:P136` for Genre, `wdt:P577` for Publication Date, `wdt:P495` for Country of Origin, `wdt:P2047` for Duration, and `wdt:P166` for Awards). Additionally, five specific properties were mapped for human entities like actors and directors (e.g., `wdt:P27` for Nationality, `wdt:P569` for Birth Date).

By dynamically injecting this strict JSON mapping directly into the LLM's system prompt and explicitly forbidding the use of any unlisted properties, the system effectively constrains the LLM's vast search space. This architectural constraint is the primary mechanism by which semantic hallucinations are entirely eradicated from the query generation process.

5. The Agentic Reflexion Loop and Error Correction

Arguably the most innovative, complex, and impactful aspect of the system’s implementation is the agent’s autonomous, self-healing error-correction mechanism, known as the *Reflexion Loop*. Translating fluid, ambiguous natural language into the strict, unforgiving, and strongly-typed syntax of SPARQL is a heavily error-prone task for any LLM, regardless of its size.

The execution flow of the agent operates in a sophisticated multi-step pipeline:

1. **Entity Extraction and Coreference Resolution:** The system first isolates the primary subject of the user’s query using a highly restrictive prompt. Crucially, it analyzes the recent conversational history to resolve anaphoric references (e.g., accurately identifying the subject when a user asks “And what country was he born in?” immediately following a discussion about Christopher Nolan).
2. **Local ID Resolution:** The extracted natural language entity is queried against the local Blazegraph instance to retrieve its exact Wikidata identifier (e.g., mapping “The Matrix” to `wd:Q83495`). This step operates entirely locally, avoiding external API calls.
3. **Chain-of-Thought SPARQL Generation:** The system constructs a complex prompt containing the user’s question, the resolved entity ID, and the strict JSON property dictionary. The LLM is instructed to reason step-by-step before outputting the final SPARQL code.
4. **Execution and Autonomous Correction:** The generated SPARQL is executed against Blazegraph. In a traditional system, a syntax error (such as a missing period, an incorrect prefix, or an invalid property) would result in a catastrophic failure and an error message shown to the user. In this agentic architecture, the runtime exception is intercepted. The system captures the raw database *trace-back* and feeds it back to the LLM in a specialized “reflexion prompt” alongside its original faulty query. The prompt explicitly instructs the model: “!ATTENTION! Your previous attempt failed. Here is the error... Correct the SPARQL to solve this problem.”

This iterative self-correction loop—allowed to run for a maximum of three attempts—mimics human debugging. It drastically increases the overall robustness, reliability, and success rate of the system, gracefully handling edge cases and syntax anomalies without human intervention.

6. Quantitative Evaluation and Model Comparison

To objectively and quantitatively validate the system’s performance, viability, and the true impact of the reflexion loop, a comprehensive automated evaluation framework

was engineered (`evaluacion.py`). A rigorous, heavily curated dataset of 103 complex natural language queries (`dataset_evaluacion.json`) was created. This dataset comprehensively covers a wide array of semantic relationships, demanding both *single-hop* queries (direct properties of an entity) and complex *multi-hop* queries (requiring graph traversal, such as finding the birthplace of a specific movie’s director).

The framework evaluated six different local LLMs (Llama 3, Llama 3.2:1B, Mistral, Gemma 2:9B, Gemma 2:2B, and DeepSeek-R1:8B) across three distinct phases: Entity Extraction Precision, SPARQL Generation Precision, and Final Response Accuracy.

The empirical results were highly revealing. The models with roughly 8 to 9 billion parameters, specifically **Llama 3** and **Gemma 2:9B**, demonstrated outstanding and nearly identical performance. Both achieved a remarkable **97.1% accuracy** in generating valid, structurally perfect SPARQL queries, leading to a final end-to-end response accuracy of over 93.7%. This proves that mid-sized, locally hosted LLMs possess the necessary logical reasoning capabilities to navigate complex ontologies when properly constrained by strict prompting.

Conversely, significantly smaller models struggled heavily. **Llama 3.2:1B** achieved a 0.0% final accuracy, completely failing to retain the complex ontology rules within its limited parameter space.

Latency and inference times were also critically evaluated. Llama 3 proved its ideal suitability for real-time conversational AI applications by achieving a highly competitive average execution time of just **9.51 seconds per question**. In stark contrast, models that failed frequently (like Llama 3.2:1B) averaged 76.78 seconds per question. This extreme latency was not due to slow inference speeds, but rather because their constant syntax errors continuously triggered the Reflexion Loop, forcing the system to exhaust all its automatic retry attempts before ultimately failing.

Finally, an ablation study quantified the precise impact of the autonomous Reflexion Loop. Comparing *Zero-Shot* accuracy (first attempt success) against *Multi-Turn* accuracy (success after self-correction) revealed significant improvements. The most dramatic impact was observed in the **Gemma 2:2B** model, which jumped from a 60.2% Zero-Shot accuracy to a **74.8% final accuracy** solely due to the agent’s ability to read database error messages and rewrite its own code. This conclusively demonstrates that the iterative agentic architecture successfully compensates for the reasoning limitations of smaller models and maximizes the reliability of more capable ones.

7. Conclusions and Future Work

The empirical results and successful implementation of this Final Degree Project confirm beyond doubt that integrating powerful LangChain-inspired agentic workflows with strongly-typed, local Knowledge Graphs is a highly viable, superior, and production-ready technical solution for building transparent Question-Answering systems. The developed chatbot successfully and completely hides the steep learning curve

and underlying complexity of the Semantic Web from the end-user, providing a fluid, conversational interface while simultaneously retaining the absolute factual precision of RDF data and eliminating LLM hallucinations.

While the current prototype achieves state-of-the-art accuracy within its domain, the rapid evolution of Natural Language Processing presents several exciting avenues for future work. First, improving the entity disambiguation phase is a priority; migrating from strict string-matching algorithms to semantic similarity searches using vector embeddings will greatly enhance robustness against user typos and idiomatic phrasing. Second, a controlled expansion of the underlying ontology to include more complex data points (such as detailed box office financials, soundtrack metadata, and critical reception scores) will allow for even richer interactions. Third, implementing dynamic *Few-Shot* prompting mechanisms will further optimize the LLM's performance on highly complex, multi-hop reasoning questions.

Finally, regarding the software architecture, migrating the presentation layer from Streamlit to a modern, fully reactive web framework (such as Vue.js or Next.js) backed by a dedicated RESTful API built in FastAPI, would significantly enhance the system's scalability, customization capabilities, and overall User Experience (UX), paving the way for a large-scale, multi-user deployment of this technology.

Keywords: Semantic Web, Knowledge Graphs, SPARQL, Large Language Models, Llama 3, RAG, GraphRAG, LangChain, Agentic AI, Hallucination Mitigation, Streamlit, Autonomous Agents.

Índice general

1. Introducción	1
1.1. Motivación	1
1.2. Objetivos	2
1.3. Estructura de la Memoria	2
2. Estado del Arte	4
2.1. Web Semántica y Grafos de Conocimiento	4
2.1.1. Resource Description Framework (RDF)	4
2.1.2. SPARQL	4
2.1.3. Wikidata	4
2.2. Modelos de Lenguaje de Gran Escala (LLMs)	5
2.3. Generación Aumentada por Recuperación (RAG)	6
2.4. Agentes Autónomos y LangChain	6
3. Análisis de objetivos y metodología	8
3.1. Metodología de Desarrollo	8
3.2. Análisis de Requisitos	8
3.2.1. Requisitos Funcionales	8
3.2.2. Requisitos No Funcionales	9
3.3. Casos de Uso	10
4. Diseño y resolución del trabajo realizado	12
4.1. Aplicación de la Metodología	12
4.2. Arquitectura del Sistema	14
4.3. Desarrollo de implementación	14
4.3.1. Interfaz de Usuario	14
4.3.2. Capa de Lógica de Negocio	15
4.3.3. Capa de Datos	17
4.4. Diseño del Agente Autocorrector	18
4.5. Diseño del Grafo de Conocimiento Local	20
5. Desarrollo e Implementación	22
5.1. Entorno de Desarrollo y Tecnologías	22
5.2. Extracción de Datos y Grafo Local	22
5.3. El Agente Conversacional y el Control del LLM	23
5.4. Bucle de Reflexión y Autocorrección	23
5.5. Interfaz Gráfica	24
6. Evaluación y Resultados	25
6.1. Entorno de Pruebas	25
6.2. Resultados Obtenidos	25
6.3. Análisis y Discusión de los Resultados	26
6.4. Conclusiones de la Evaluación	28

7. Conclusiones y Trabajo Futuro	29
7.1. Consecución de Objetivos	29
7.2. Líneas de Trabajo Futuro	29
Bibliografía	31
Lista de Acrónimos y Abreviaturas	32
A. Manual de Instalación y Despliegue	33
A.1. Requisitos Previos	33
A.2. Despliegue de Blazegraph	33
A.3. Configuración de Ollama y Modelos Locales	33
A.4. Configuración del Entorno Python	34
A.5. Ejecución de la Interfaz (Streamlit)	34

Índice de figuras

2.1.	Ejemplo de grafo de conocimiento sobre el dominio del cine.	5
3.1.	Diagrama de Secuencia Principal	10
4.1.	Arquitectura del sistema	15
4.2.	Interfaz de usuario en modo oscuro	16
4.3.	Interfaz de usuario en modo claro	17
4.4.	Respuesta del sistema a una consulta.	18
4.5.	Diagrama de Secuencia con Bucle de Autocorrección	19
4.6.	Respuesta del sistema con el bucle de autocorrección activado. . .	20

Índice de tablas

6.1.	Comparativa de rendimiento, precisión y latencia entre distintos LLMs locales evaluados.	25
6.2.	Impacto del Bucle de Reflexión sobre la precisión de generación SPARQL.	26

Índice de Códigos

5.1. Carga del mapeo de propiedades e inyección en el prompt SPARQL . .	23
5.2. Bucle de reflexión ante errores de Blazegraph	23

1. Introducción

En la actualidad, el acceso a grandes volúmenes de información estructurada y no estructurada se ha convertido en un desafío fundamental en el ámbito de las Ciencias de la Computación. En el contexto de la Web Semántica, los Grafos de Conocimiento (*Knowledge Graphs*) ofrecen un marco robusto para representar información mediante tripletas (Sujeto, Predicado, Objeto), permitiendo realizar consultas complejas mediante el lenguaje estándar SPARQL. Sin embargo, la barrera técnica que supone dominar este lenguaje limita el acceso a esta información para usuarios no expertos.

Por otro lado, el auge de los Modelos de Lenguaje de Gran Escala (LLMs) ha transformado la forma en que interactuamos con los sistemas de información, permitiendo interfaces conversacionales en lenguaje natural. No obstante, los LLMs sufren de problemas inherentes como las alucinaciones (generación de información plausible pero falsa) y la dificultad para recuperar datos precisos y actualizados de dominios específicos.

Este Trabajo de Fin de Grado (TFG) aborda la convergencia de ambas tecnologías mediante el desarrollo de un agente conversacional (chatbot) especializado en el dominio cinematográfico. El sistema combina la fluidez y capacidad de comprensión del lenguaje natural de los LLMs con la precisión de un Grafo de Conocimiento propio basado en Wikidata.

Para lograrlo, se ha diseñado un agente capaz de interpretar la intención del usuario, mapear entidades a identificadores semánticos y generar autónomamente consultas SPARQL válidas, pudiendo llegar a corregirse a sí mismo en caso de error gracias a un bucle de reflexión.

1.1. Motivación

La motivación principal de este proyecto surge de la necesidad de facilitar la exploración de datos semánticos sin requerir conocimientos técnicos previos. Extraer información concreta de un grafo RDF con millones de nodos (como es el caso del dominio del cine en Wikidata) resulta una tarea ardua.

Si bien los LLMs pueden responder preguntas de cultura general sobre cine, su conocimiento es estático y propenso a errores. Integrar un LLM como interfaz a un Grafo de Conocimiento mediante una arquitectura basada en la generación de SPARQL no solo asegura la veracidad de la respuesta, sino que permite trazar el origen del dato y justificar la respuesta.

Además, este proyecto busca explorar la complejidad técnica que supone dominar y restringir la salida de un LLM. Lograr que un modelo de lenguaje actúe no como un mero generador de texto libre, sino como un agente lógico que respeta esquemas estrictos (ontologías) y corrige iterativamente su propio código SPARQL ante errores sintácticos, representa un reto de ingeniería de software y de *prompt engineering* altamente motivador.

1.2. Objetivos

El objetivo general de este proyecto es diseñar, desarrollar y evaluar un sistema de pregunta-respuesta (QA) conversacional basado en Grafos de Conocimiento capaz de traducir consultas en lenguaje natural a consultas SPARQL precisas y ejecutables sobre una base de datos propia.

Para alcanzar este fin, se definen los siguientes objetivos específicos:

- **Precisión en la Generación de SPARQL:** Desarrollar un agente autónomo que maximice la precisión al traducir lenguaje natural a SPARQL, siendo capaz de gestionar un bucle de autocorrección (reflexión) para enmendar errores sintácticos o semánticos generados en intentos previos.
- **Manejo Avanzado de Datos RDF:** Extraer, procesar y almacenar un conjunto significativo de datos provenientes de Wikidata en formato *N-Triples* para conformar un Grafo de Conocimiento local optimizado, demostrando soltura en el manejo de tecnologías de la Web Semántica.
- **Control y Restricción del LLM:** Diseñar un marco robusto de *prompting* estructurado y mapeo de propiedades que permita "restringir" y guiar el razonamiento del modelo de lenguaje, minimizando las alucinaciones y forzándolo a utilizar únicamente las propiedades y entidades definidas en el contexto.
- **Evaluación Cuantitativa:** Implementar un marco de evaluación automática que permita medir objetivamente el rendimiento del sistema en términos de extracción de entidades correctas y generación de consultas SPARQL válidas frente a un conjunto de pruebas.

1.3. Estructura de la Memoria

El resto de esta memoria se estructura de la siguiente manera:

En el **Capítulo 2** se presenta el Estado del Arte, donde se exploran los fundamentos teóricos de la Web Semántica, los Modelos de Lenguaje y las arquitecturas de Generación Aumentada por Recuperación (RAG) e Inteligencia Artificial Agéntica.

El **Capítulo 3** detalla el Análisis del sistema, describiendo los requisitos técnicos y los casos de uso principales.

El **Capítulo 4** expone el Diseño de la solución, abordando la arquitectura del sistema, el diseño del Grafo de Conocimiento y el flujo de ejecución del agente iterativo.

El **Capítulo 5** describe la Implementación del proyecto, profundizando en las decisiones de programación, el entorno tecnológico utilizado (Python, Ollama, Streamlit) y el despliegue del sistema mediante Docker.

En el **Capítulo 6** se presenta la Evaluación del sistema, analizando las métricas obtenidas tras someter al agente a diversas baterías de pruebas automatizadas.

Finalmente, el **Capítulo 7** recoge las Conclusiones extraídas del desarrollo, evaluando el grado de cumplimiento de los objetivos y proponiendo líneas de trabajo futuro.

2. Estado del Arte

Para comprender la arquitectura y las decisiones de diseño adoptadas en este Trabajo de Fin de Grado, es necesario establecer una base teórica sobre las tecnologías subyacentes. Este capítulo revisa los conceptos clave en los que se apoya el sistema desarrollado: la Web Semántica, los Modelos de Lenguaje de Gran Escala (LLMs) y los paradigmas emergentes de Inteligencia Artificial como la Generación Aumentada por Recuperación (RAG) y los agentes autónomos.

2.1. Web Semántica y Grafos de Conocimiento

La Web Semántica, propuesta originalmente por Tim Berners-Lee, busca extender la web actual dotando a la información de un significado bien definido, permitiendo que ordenadores y personas trabajen en cooperación. El núcleo de esta visión son los Grafos de Conocimiento (*Knowledge Graphs*), estructuras de datos que representan entidades del mundo real y sus interrelaciones de forma explícita.

2.1.1. Resource Description Framework (RDF)

RDF es el estándar principal para modelar información en la Web Semántica [1]. La información se estructura en forma de tripletas que representan una relación entre dos entidades, por lo que el modelo de datos está estructurado en forma de grafo, donde cada nodo es una entidad y cada arista es una relación entre dos entidades: *Sujeto*, *Predicado* y *Objeto*.

Estas tripletas pueden serializarse en distintos formatos, siendo *nt* (N-Triples) uno de los más eficientes para el almacenamiento, tiempo de procesamiento y consultas complejas que requieran pasar de unas entidades a otras (multi-hop queries), como se muestra en la siguiente figura.

2.1.2. SPARQL

SPARQL (*SPARQL Protocol and RDF Query Language*) es el lenguaje de consulta estándar para bases de datos RDF [2]. Permite realizar consultas complejas sobre grafos, filtrar resultados y explorar las relaciones semánticas entre entidades. Aunque es extremadamente potente y preciso, su curva de aprendizaje es muy pronunciada, lo que justifica la necesidad de construir interfaces en lenguaje natural para democratizar el acceso a los datos, como se propone en este TFG.

2.1.3. Wikidata

Wikidata es una base de conocimiento libre, colaborativa y multilingüe que actúa como almacenamiento central para los datos estructurados de sus proyectos hermanos, como Wikipedia [3]. Dado su enorme volumen de datos y su estructura semántica abierta y en constante evolución, Wikidata se ha consolidado como una de las fuentes

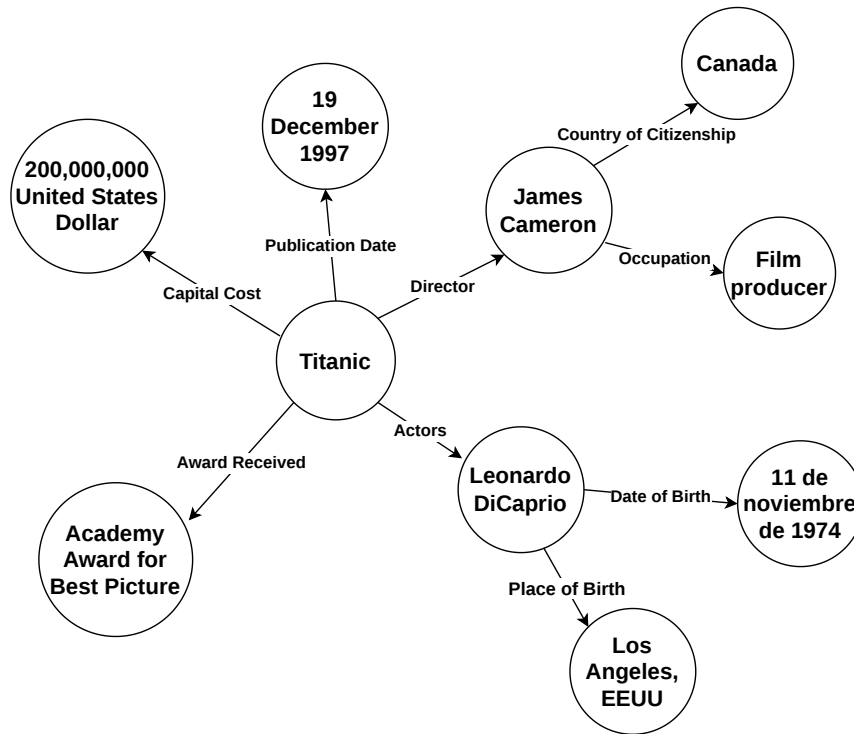


Figura 2.1: Ejemplo de grafo de conocimiento sobre el dominio del cine.

de conocimiento general más importantes del mundo, sirviendo como origen de los datos sobre la industria del cine extraídos para este proyecto.

2.2. Modelos de Lenguaje de Gran Escala (LLMs)

Los Modelos de Lenguaje de Gran Escala (LLMs), como la familia GPT de OpenAI o los modelos LLaMA de Meta, son redes neuronales masivas basadas en la arquitectura *Transformer* [4], entrenadas sobre cantidades ingentes de texto. Estos modelos han demostrado capacidades sin precedentes en la comprensión, síntesis y generación de lenguaje natural.

Sin embargo, en el contexto de la recuperación de información real y precisa, los LLMs presentan dos limitaciones críticas inherentes a su naturaleza estocástica:

1. **Conocimiento estático:** El modelo solo conoce la información disponible hasta el momento de su entrenamiento, requiriendo reentrenamientos costosos para actualizarse.
2. **Alucinaciones:** Tendencia a generar respuestas altamente plausibles a nivel sintáctico pero incorrectas cuando el modelo no dispone de la información requerida

en sus pesos internos.

Para mitigar estos problemas, especialmente en aplicaciones de dominio específico, es estrictamente necesario acoplar el LLM con fuentes de conocimiento externas fidedignas.

2.3. Generación Aumentada por Recuperación (RAG)

La Generación Aumentada por Recuperación (*Retrieval-Augmented Generation*, RAG) es una arquitectura [5] que mejora sustancialmente la fiabilidad de las respuestas de un LLM al integrar un sistema de recuperación de información y al obligar al LLM a basar sus respuestas en la información específica proporcionada en el momento de la consulta. En un flujo RAG estándar:

1. El usuario realiza una pregunta en lenguaje natural.
2. El sistema busca información relevante en una base de datos externa.
3. La información recuperada se inyecta en el *prompt* (contexto) del LLM.
4. El LLM formula la respuesta final basándose en el contexto documentado proporcionado, en lugar de depender únicamente de su memoria interna.

En este TFG, se aplica una variante más estructurada de RAG, a menudo referida como **Graph RAG** o **Text-to-SPARQL RAG**. En lugar de recuperar fragmentos de texto mediante similitud vectorial (que puede ser imprecisa), el modelo se encarga de generar el código lógico (SPARQL) necesario para interrogar un Grafo de Conocimiento de forma determinista, garantizando respuestas estructuradas y exactas.

2.4. Agentes Autónomos y LangChain

Un paso más allá del uso estático de LLMs es el paradigma de la Inteligencia Artificial Agéntica. Un agente de IA es un sistema que utiliza un LLM como su "motor de razonamiento" central para decidir dinámicamente qué acciones tomar, qué herramientas utilizar y en qué orden.

A diferencia de un flujo de código secuencial clásico, un agente puede observar el resultado de una acción y reaccionar. Por ejemplo, si el agente genera una consulta SPARQL que, al ejecutarse en el motor de base de datos, devuelve un error de sintaxis, el agente puede leer dicho error, reflexionar sobre el fallo, autocorregir el código y volver a intentar la consulta. Este bucle de retroalimentación dota al sistema de una robustez crítica frente a la variabilidad de las respuestas del modelo.

LangChain [6] es el *framework* de desarrollo de código abierto líder en el ecosistema para la creación de aplicaciones impulsadas por modelos de lenguaje. Proporciona

las abstracciones necesarias para construir cadenas de procesamiento complejas, integrar herramientas externas, gestionar la memoria de las conversaciones y orquestar el flujo cognitivo de agentes autónomos orientados a la recuperación de conocimiento estructurado.

3. Análisis de objetivos y metodología

Este capítulo detalla el análisis previo a la implementación del sistema, describiendo la metodología de trabajo adoptada y los requisitos funcionales y no funcionales que han guiado el desarrollo del chatbot basado en Grafos de Conocimiento.

3.1. Metodología de Desarrollo

Dada la naturaleza exploratoria de este Trabajo de Fin de Grado, centrado en la integración de Inteligencia Artificial Generativa y Web Semántica, la adopción de una metodología clásica en cascada (*Waterfall*) resultaba inapropiada. El comportamiento de los Modelos de Lenguaje de Gran Escala (LLMs) es no determinista y requiere de experimentación continua, especialmente en el ámbito de la ingeniería de *prompts* y la extracción de datos.

Por ello, se ha optado por un **enfoque ágil e iterativo basado en prototipos**. El desarrollo se ha dividido en ciclos cortos de experimentación:

1. **Exploración y Extracción de Datos:** Pruebas iniciales para consultar Wikidata y consolidar un Grafo de Conocimiento local funcional y acotado.
2. **Prototipado del Agente:** Desarrollo iterativo del *prompt* principal del LLM para traducir lenguaje natural a SPARQL.
3. **Implementación del Bucle de Reflexión:** Adición del mecanismo de autocorrección ante errores de sintaxis en el código generado.
4. **Evaluación y Refinamiento:** Pruebas de rendimiento frente al *dataset* de evaluación que derivaron en ajustes continuos en el flujo de ejecución.
5. **Implementación del Frontend:** Desarrollo de un interfaz de usuario para la interacción con el sistema.

Este enfoque ha permitido volver a pasos anteriores en caso de que fuera necesario, permitiendo un mayor desarrollo de las estrategias de *prompting* y una mayor integración de los componentes del sistema.

3.2. Análisis de Requisitos

Para asegurar que el sistema cumple con los objetivos planteados, se han especificado una serie de requisitos funcionales y no funcionales.

3.2.1. Requisitos Funcionales

Los requisitos funcionales (RF) describen las interacciones y el comportamiento esperado del sistema:

- **RF-01. Interfaz conversacional:** El sistema debe proporcionar una interfaz gráfica que permita al usuario realizar preguntas en lenguaje natural de forma intuitiva, similar a un chat convencional.
- **RF-02. Generación de la base de conocimiento local:** El sistema debe disponer de un módulo para extraer entidades de Wikidata.
- **RF-03. Extracción y desambiguación de entidades:** El sistema debe ser capaz de identificar la entidad principal en la consulta del usuario (ej. nombre de un actor o película) para obtener su identificador único (ej. `wd:Q25191`).
- **RF-04. Generación de SPARQL:** El agente LLM debe traducir la pregunta del usuario en una consulta SPARQL sintáctica y semánticamente válida, usando la entidad extraída y el diccionario de propiedades del sistema.
- **RF-05. Obtención de resultados:** El sistema debe ser capaz de obtener los resultados de la consulta SPARQL a partir de la base de datos local.
- **RF-06. Bucle de autocorrección:** Si la consulta SPARQL generada produce un error al ejecutarse en el motor RDF, el agente debe leer el mensaje de error y su propia consulta para intentar corregirla de forma autónoma.
- **RF-07. Ejecución y respuesta:** El sistema debe presentar la respuesta extraída al usuario en un formato legible.
- **RF-08. Mantenimiento del contexto:** El sistema debe ser capaz de resolver correferencias, para poder responder a preguntas como "¿Qué otras películas ha protagonizado?" sin necesidad de repetir el nombre del actor.
- **RF-09. Transparencia y Explicabilidad:** El sistema debe mostrar al usuario el razonamiento interno, la detección de entidades y el código SPARQL generado en tiempo real.

3.2.2. Requisitos No Funcionales

Los requisitos no funcionales (RNF) definen las restricciones y atributos de calidad del sistema:

- **RNF-01. Precisión y mitigación de alucinaciones:** El sistema debe priorizar la exactitud del código SPARQL generado, limitando estrictamente el alcance de conocimiento del LLM al esquema de la base de datos local para evitar alucinaciones.
 - **RNF-02. Privacidad y Ejecución Local:** El sistema debe garantizar la total privacidad de las consultas y la independencia de APIs externas de pago o en la nube para funcionar.
-

- **RNF-03. Tiempos de respuesta:** El sistema debe mantener tiempos de respuesta aceptables (generalmente inferiores a los 10 segundos), teniendo en cuenta el tiempo requerido para las llamadas a la API del LLM y la desambiguación.
- **RNF-04. Modularidad:** El código debe estar desacoplado, separando la lógica de la interfaz (Streamlit), la lógica del agente (Ollama) y la extracción de datos (Wikidata).
- **RNF-05. Escalabilidad:** El sistema debe permitir la ampliación del dominio de conocimiento modificando únicamente el fichero de configuración JSON, sin necesidad de reprogramar la lógica central del agente

3.3. Casos de Uso

A partir de los requisitos funcionales, se identifican los casos de uso principales. El caso de uso central del sistema es la *Consulta de información cinematográfica*.

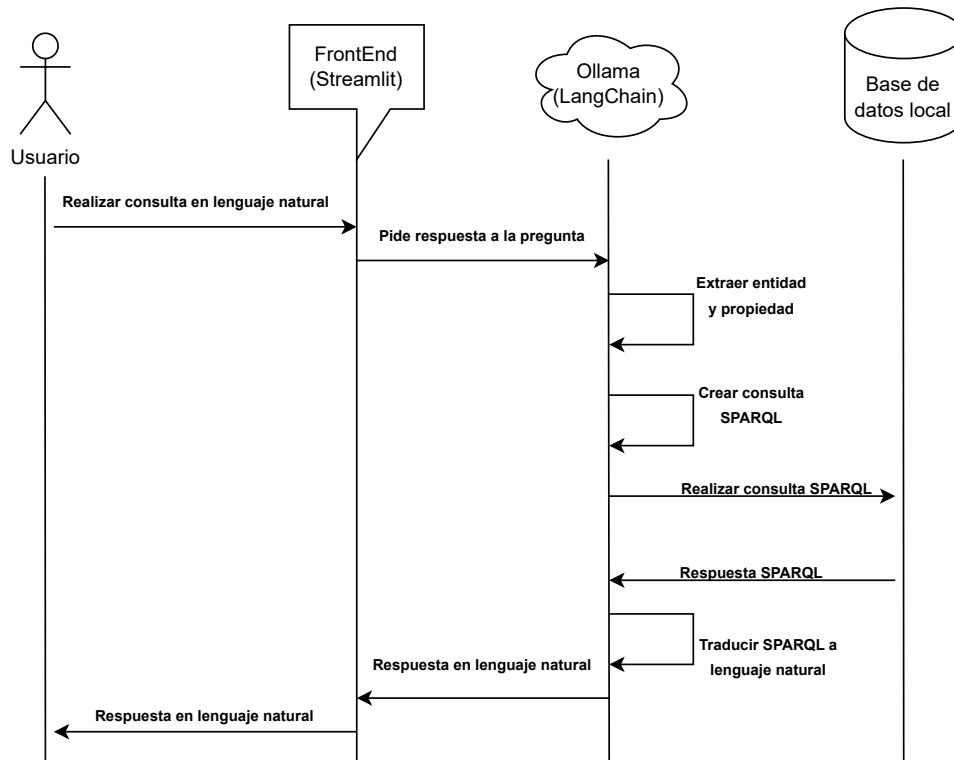


Figura 3.1: Diagrama de Secuencia Principal

En este caso de uso central, el Actor (Usuario) interactúa con la interfaz de chat. El

sistema, de forma transparente para el usuario, ejecuta sub-casos como la extracción de entidades, la generación de código SPARQLy la consulta a la base de datos local.

4. Diseño y resolución del trabajo realizado

En este capítulo se expone la arquitectura técnica del sistema, el diseño del Grafo de Conocimiento y el modelado del comportamiento del agente inteligente responsable de traducir el lenguaje natural a consultas SPARQL.

4.1. Aplicación de la Metodología

Tal como se explicó en el apartado anterior, la metodología seguida combina un enfoque iterativo con la creación de prototipos y la validación continua mediante experimentación de *prompts* y modelos. Cada uno de los pasos descritos —Exploración y Extracción de Datos, Prototipado del Agente, Implementación del Bucle de Reflexión, Evaluación y Refinamiento, e Implementación del Frontend— se ha orientado a cubrir las necesidades clave identificadas durante el análisis. A continuación, se muestra cómo cada aspecto metodológico responde a esas necesidades concretas:

1. Gestión estructurada del conocimiento y prevención de alucinaciones

- **Análisis de Requisitos:** Se determinó la necesidad de disponer de una base de conocimiento estructurada y acotada para evitar que el LLM inventara datos (alucinaciones).
- **Prototipo inicial:** Se realizaron consultas directas a la API de Wikidata para validar la viabilidad de la extracción de entidades y el modelado de la información.
- **Diseño del Sistema:** Se optó por la creación de un Grafo de Conocimiento local (archivo `.nt`) gestionado con *Blazegraph* para garantizar consistencia semántica y tiempos de respuesta rápidos.
- **Desarrollo e Implementación:** Se implementó un esquema estricto mediante un diccionario de propiedades, limitando el espacio de búsqueda del modelo de lenguaje únicamente a la ontología definida.

2. Precisión y resiliencia ante errores de sintaxis

- **Análisis de Requisitos:** Se identificó la inestabilidad inherente de los LLMs a la hora de generar código estricto, como es el caso del lenguaje de consultas SPARQL.
- **Diseño del Sistema:** Se diseñó un flujo de control basado en un modelo de agente iterativo, priorizando la capacidad de reflexión sobre los fallos.
- **Desarrollo e Implementación:** Se implementó directamente sobre *Ollama* un Bucle de Autocorrección: si la base de datos devuelve un error, el sistema no colapsa; en su lugar, captura el mensaje de error y solicita al agente que corrija su propia consulta de forma autónoma.

- **Pruebas y Validación:** Se evaluó el rendimiento del agente frente a un *dataset* validado, logrando elevar significativamente el porcentaje de acierto respecto al enfoque de intento único (*zero-shot*).

3. Privacidad e independencia de la nube (Ejecución Local)

- **Análisis de Requisitos:** Se definió la importancia de mantener la privacidad de las consultas y no depender económicamente de APIs de pago externas (como OpenAI).
- **Diseño del Sistema:** Se decidió utilizar *Ollama* como motor de inferencia principal, permitiendo ejecutar modelos de última generación (como Llama 3) de manera 100% local.
- **Desarrollo e Implementación:** El núcleo del procesamiento (extracción de entidades, traducción de lenguaje natural a SPARQL y resolución de resultados) ocurre íntegramente de forma local en la máquina del usuario, sin ninguna llamada a servicios externos durante la fase de consulta.

4. Transparencia y Explicabilidad

- **Análisis de Requisitos:** Se reconoció la necesidad de que el usuario no viera al sistema como una "caja negra", sino que pudiera comprender cómo se ha obtenido una respuesta.
- **Diseño del Sistema:** Se planificó la incorporación de características de Inteligencia Artificial Explicable (XAI) directamente en la interfaz.
- **Desarrollo e Implementación:** Se configuró el sistema para exponer en tiempo real los pasos del agente: desde la identificación de la entidad principal, pasando por los intentos de generación de código SPARQL, hasta la ejecución contra el repositorio local.

5. Experiencia de usuario conversacional

- **Análisis de Requisitos:** Desde el inicio se priorizó la usabilidad, buscando que usuarios sin conocimientos técnicos sobre Grafos de Conocimiento pudieran interactuar de manera fluida.
- **Diseño del Sistema:** Se seleccionó *Streamlit* como *framework* para el *frontend*, dada su agilidad para prototipar interfaces de datos en Python.
- **Pruebas y Validación:** Se refinó la interfaz de chat, ajustando el historial de mensajes y la resolución de correferencias para permitir conversaciones naturales (ej. preguntar "Quién la dirigió" justo después de mencionar una película).

6. Visión de futuro, modularidad y extensibilidad

- **Análisis de Requisitos:** Se previó la necesidad de ampliar el dominio del chatbot a otros campos más allá del cine o evaluar el rendimiento con modelos futuros.
- **Diseño del Sistema:** Se adoptó una arquitectura por capas altamente desacoplada (Presentación, Lógica del Agente y Capa de Datos).
- **Desarrollo e Implementación:** Al parametrizar el modelo LLM mediante *Ollama* y externalizar el esquema de propiedades en un fichero JSON de configuración, se garantiza que el sistema puede integrar nuevos modelos, ontologías o grafos sin necesidad de modificar la lógica central del agente.

4.2. Arquitectura del Sistema

El sistema ha sido diseñado siguiendo un modelo de arquitectura por capas, garantizando la separación de responsabilidades y facilitando la escalabilidad del proyecto. Los tres bloques principales son:

1. **Capa de Interfaz de Usuario:** Implementada con *Streamlit*, actúa como el cliente de usuario, proveyendo la interfaz de chat interactiva.
2. **Capa de Lógica de Negocio:** Es el núcleo del sistema, implementado en Python con *Ollama* como motor de inferencia. Contiene la lógica de extracción de entidades, la integración con el LLM local, el bucle de reflexión y la construcción de los *prompts* del sistema.
3. **Capa de Datos:** Comprende el almacenamiento persistente RDF en un archivo *.nt* gestionado por *Blazegraph* como motor SPARQL local, accedido mediante la librería *SPARQLWrapper*.

4.3. Desarrollo de implementación

A continuación, se detallan todas las decisiones de diseño e implementación que se han tomado durante el desarrollo del proyecto, siguiendo la arquitectura anteriormente descrita. Asimismo, se menciona como cada módulo se integra y comunica con los demás.

4.3.1. Interfaz de Usuario

El frontend de la aplicación ha sido desarrollado utilizando *Streamlit*, una biblioteca de Python que actúa como framework web. Este software destaca por su equilibrio entre sencillez y potencia, resultando adecuado para este tipo de proyectos. En este

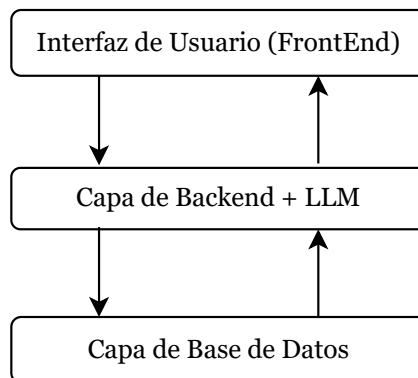


Figura 4.1: Arquitectura del sistema

proyecto se ha centrado la mayor parte del esfuerzo en la lógica negocio, por lo que un software capaz de crear interfaces de usuario de forma rápida y sencilla ha sido clave. Además, su arquitectura permite una integración sencilla con otras librerías de Python, como es el caso de *LangChain* y *Ollama*.

Además de la interacción conversacional habitual, la interfaz ha sido diseñada para proporcionar total transparencia sobre el funcionamiento interno del sistema. Durante cada consulta, se expone en la propia pantalla el proceso de razonamiento del agente, detallando paso a paso acciones clave como la extracción de la entidad solicitada, la generación y validación del código SPARQL, y la gestión de reintentos automáticos. Mostrar de forma clara este *pipeline* interno resulta fundamental en el contexto del proyecto, ya que permite comprender y validar el comportamiento de la arquitectura subyacente.

4.3.2. Capa de Lógica de Negocio

La lógica de negocio reside íntegramente en el módulo `chatLocal.py`, que actúa como orquestador del flujo de procesamiento. Este módulo se estructura en tres funciones principales que se ejecutan de forma secuencial para cada consulta recibida:

- **extraer_entidad:** Recibe la pregunta del usuario y el historial reciente de la conversación. Mediante un *prompt* altamente restrictivo, solicita al LLM que identifique y devuelva únicamente el nombre propio sobre el que versa la pregunta (película, actor, director, etc.), sin responder a ella. Esto permite resolver correferencias anafóricas del tipo “¿Y él quién la dirigió?” apoyándose en el historial.

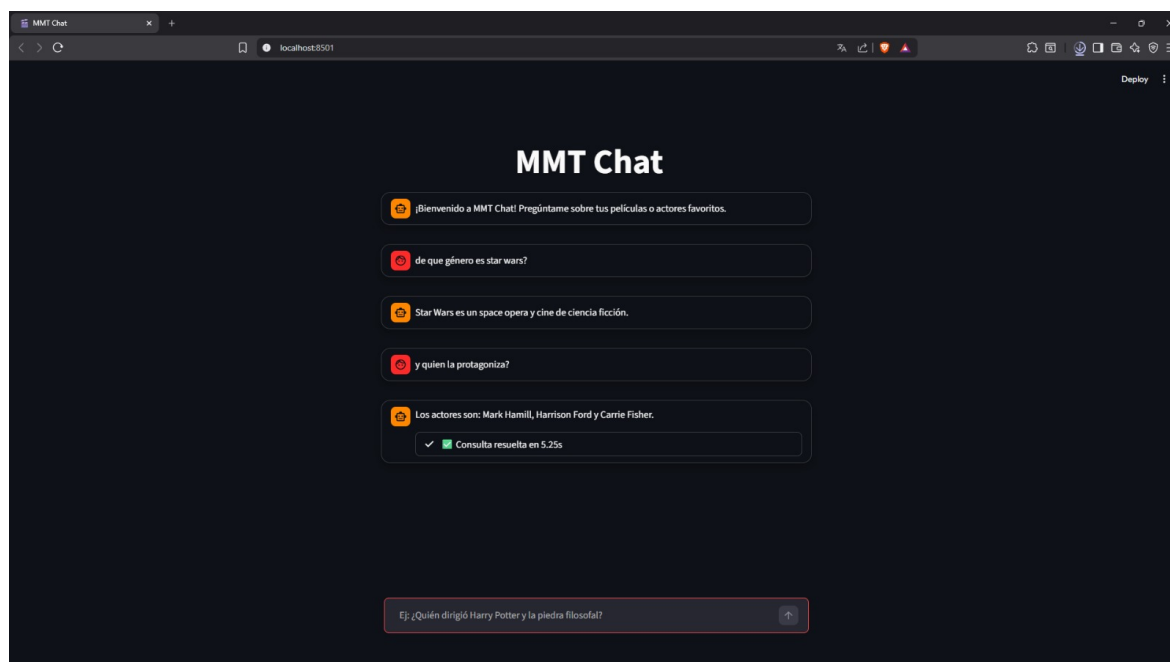


Figura 4.2: Interfaz de usuario en modo oscuro

- **generar_sparql:** Construye el *prompt* de generación SPARQL inyectando la entidad extraída, su identificador de Wikidata y el diccionario de propiedades leído desde `mapeo_propiedades.json`. Solicita al LLM que razone paso a paso (*Chain of Thought*) antes de devolver el código SPARQL dentro de un bloque delimitado, facilitando su posterior extracción con expresiones regulares. Si se le pasa un error previo, el *prompt* incluye la consulta fallida y el motivo del fallo para que el modelo pueda corregirla.
- **procesar_consulta:** Función principal que coordina el flujo completo: llama a `extraer_entidad`, resuelve el identificador Wikidata en la base de datos local mediante `buscar_id_entidad`, ejecuta el bucle iterativo de generación y corrección SPARQL (hasta tres intentos), traduce los resultados a texto legible y devuelve la respuesta formateada por el LLM.

La conexión con el LLM se realiza directamente a través de la librería `ollama`, invocando el método `ollama.chat()` con temperatura cero para maximizar el determinismo de las respuestas. Esto permite cambiar de modelo (Llama 3, Mistral, Gemma 2, etc.) simplemente modificando el parámetro correspondiente, sin alterar ninguna otra parte del sistema.

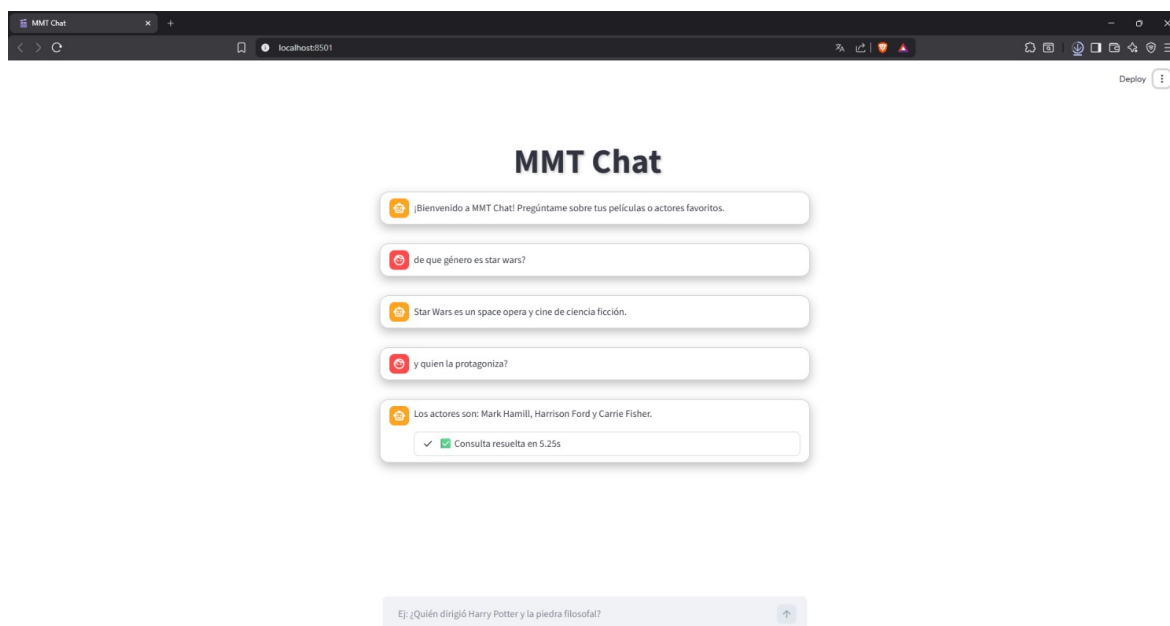


Figura 4.3: Interfaz de usuario en modo claro

4.3.3. Capa de Datos

La capa de datos se sustenta en *Blazegraph*, un motor de base de datos RDF de alto rendimiento que expone un endpoint SPARQL local en el puerto 9999. El grafo de conocimiento (archivo `.nt`) se carga una única vez en Blazegraph al inicio del proceso y permanece disponible en memoria durante toda la sesión, garantizando tiempos de respuesta rápidos ante las consultas del agente.

El acceso al endpoint desde Python se realiza mediante la librería *SPARQLWrapper*, que permite lanzar consultas SPARQL estándar y recibir los resultados en formato JSON. Este diseño presenta dos ventajas clave frente a alternativas de carga en memoria (como RDFLib): en primer lugar, Blazegraph mantiene índices internos que hacen las consultas significativamente más rápidas sobre grafos de gran tamaño; en segundo lugar, el servidor puede recargarse o ampliarse sin reiniciar la aplicación.

La resolución de entidades también opera sobre esta misma capa: `entidades_cine.py` ejecuta primero una búsqueda exacta por *label* contra Blazegraph local y, si no halla resultado, intenta una búsqueda parcial por subcadena. Este diseño elimina la dependencia de la API externa de Wikidata durante la fase de consulta, reforzando el requisito de ejecución completamente local (RNF-02).

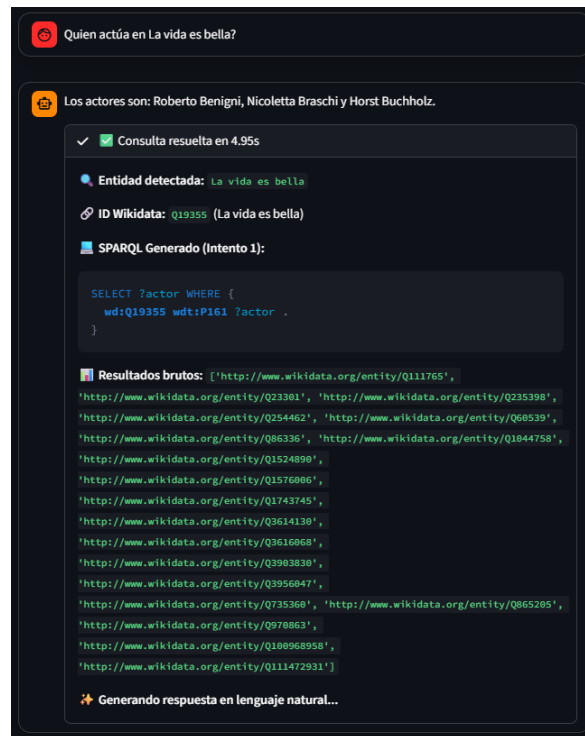


Figura 4.4: Respuesta del sistema a una consulta.

4.4. Diseño del Agente Autocorrector

Uno de los principales desafíos técnicos del proyecto radica en la inestabilidad inherente de los LLMs a la hora de generar código con sintaxis estricta, como es el caso de SPARQL. Para solucionar esto, se ha diseñado un **flujo iterativo de reflexión y autocorrección**.

El flujo de control del agente sigue los siguientes pasos lógicos:

1. Se recibe la pregunta en lenguaje natural.
2. Se solicita al LLM que extraiga únicamente la entidad principal (ej. “Matrix”), teniendo en cuenta el historial de conversación para resolver referencias anafóricas.
3. Se realiza una búsqueda local en *Blazegraph* para obtener el identificador Wikidata de dicha entidad (ej. `wd:Q83495`), sin necesidad de acceder a servicios externos.
4. Se inyecta la pregunta, el ID de la entidad y el *diccionario de propiedades* en un *prompt* altamente restrictivo.
5. El LLM genera una propuesta de consulta SPARQL.

6. El sistema intenta ejecutar la consulta localmente en Blazegraph mediante SPARQLWrapper.
7. **Bucle de corrección:** Si la consulta falla (error de sintaxis o error de acceso a la ontología), el sistema no aborta la operación. Captura el *traceback* del error, se lo envía de vuelta al LLM y le solicita que reescriba el SPARQL corrigiendo su propio fallo. Este proceso se repite un número máximo de iteraciones.

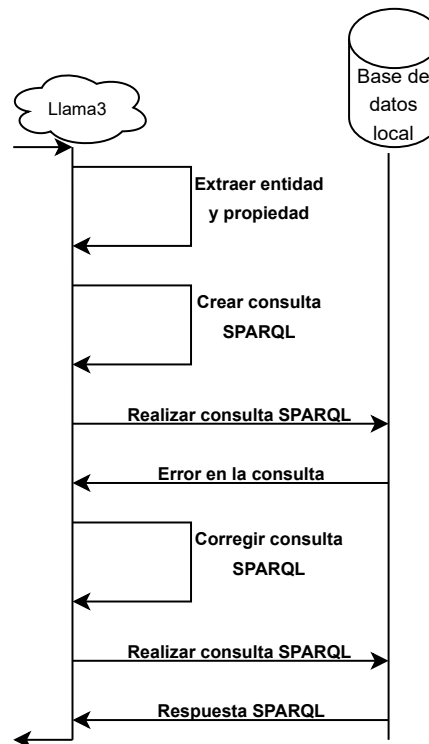


Figura 4.5: Diagrama de Secuencia con Bucle de Autocorrección

Como se menciona anteriormente, una interacción como esta también se muestra de manera transparente al usuario mediante la interfaz de la aplicación, permitiendo al usuario ver el proceso de generación SPARQL y la corrección de errores.



Figura 4.6: Respuesta del sistema con el bucle de autocorrección activado.

4.5. Diseño del Grafo de Conocimiento Local

Para asegurar la privacidad, el rendimiento y la consistencia de las respuestas, el sistema no interroga directamente a Wikidata en su fase de resolución de consultas, sino que opera sobre un Grafo de Conocimiento local exportado previamente.

Este grafo local (`datos_cine_limpio.nt`) actúa como la base de datos del sistema, gestionada por *Blazegraph*. Su esquema semántico está rigurosamente acotado para el dominio del cine: se seleccionaron doce propiedades clave de Wikidata para limitar el espacio de búsqueda del LLM y mejorar la precisión en la generación SPARQL. Las propiedades mapeadas son:

- `wdt:P57`: Director.
- `wdt:P161`: Actor o miembro del reparto.
- `wdt:P136`: Género cinematográfico.
- `wdt:P577`: Fecha de publicación o estreno.
- `wdt:P162`: Productor.

- `wdt:P345`: Identificador de IMDb.
- `wdt:P495`: País de origen.
- `wdt:P2047`: Duración.
- `wdt:P364`: Idioma original.
- `wdt:P58`: Guionista.
- `wdt:P166`: Premios recibidos.
- `wdt:P2130`: Coste o presupuesto de producción.

Además, también se han seleccionado las cinco propiedades principales de algunas de las entidades relacionadas con las películas, como son los actores y directores de estas. Estas propiedades son:

- `wdt:P27`: Nacionalidad.
- `wdt:P569`: Fecha de nacimiento.
- `wdt:P570`: Fecha de fallecimiento.
- `wdt:P106`: Profesión.
- `wdt:P19`: Lugar de nacimiento.

El LLM recibe en su *prompt* un diccionario estricto que mapea el lenguaje natural a estas propiedades —leído dinámicamente desde `mapeo_propiedades.json`—, con la orden explícita de no utilizar ninguna propiedad de Wikidata que no se encuentre en dicha lista. Esta decisión de diseño es crítica para evitar alucinaciones semánticas y obtener los altos porcentajes de precisión en la generación de SPARQL que se documentan en el capítulo de evaluación.

5. Desarrollo e Implementación

En este capítulo se detallan los aspectos técnicos de la implementación del sistema. Se exponen las decisiones de programación, las herramientas utilizadas y fragmentos de código clave que ilustran la lógica interna del chatbot.

5.1. Entorno de Desarrollo y Tecnologías

El sistema ha sido desarrollado íntegramente en Python, aprovechando su extenso ecosistema para el procesamiento de datos y la Inteligencia Artificial. Entre las librerías principales destacan:

- **Ollama:** Motor de inferencia local que permite ejecutar LLMs de código abierto (Llama 3, Mistral, Gemma 2, etc.) directamente en la máquina del usuario sin depender de APIs externas. Se accede desde Python mediante la librería cliente oficial `ollama`.
- **Blazegraph:** Base de datos RDF de alto rendimiento que actúa como motor SPARQL local, sobre la que se carga el grafo de conocimiento en formato *N-Triples*.
- **Streamlit:** Para la rápida construcción de la interfaz gráfica de usuario.
- **SPARQLWrapper:** Para lanzar consultas SPARQL contra Blazegraph local y, durante la fase de extracción de datos, contra Wikidata.

El despliegue del sistema se ha contenerizado utilizando **Docker**, garantizando así que el entorno de ejecución sea reproducible en cualquier máquina sin problemas de dependencias.

5.2. Extracción de Datos y Grafo Local

Para evitar depender de llamadas externas costosas durante la interacción con los usuarios, se diseñó el proceso `extraer_dump_cine.py` que procesa el volcado (dump) masivo de Wikidata en local y guarda la información relevante en el archivo `datos_cine_limpio.nt`, que funciona como base de datos local, las propiedades extraídas de las entidades relevantes son las presentes en `mapeo_propiedades.json`.

Este JSON fue diseñado, además de para mapear las propiedades de las entidades deseadas del grafo de Wikidata, para que el LLM pueda entender mejor el contexto de las entidades y generar consultas SPARQL más precisas.

La estructura del código permite añadir entidades relevantes al grafo local. Por ejemplo, al extraer una película, también se extraen los nodos de su director y actores, conformando un subgrafo fuertemente conectado, ideal para responder a las consultas SPARQL posteriores.

5.3. El Agente Conversacional y el Control del LLM

El componente principal, ubicado en `chatLocal.py`, inicializa la conexión con el motor Ollama y funciona como el cerebro del sistema, encargándose de toda la lógica de la aplicación. El aspecto más crítico de la implementación en esta fase es el diseño de los *Prompts* del sistema, ya que son los que guían al LLM en la generación de consultas SPARQL y en la respuesta a las preguntas de los usuarios.

Para asegurar que el modelo utilice exclusivamente las propiedades permitidas en el grafo local, se inyecta dinámicamente el JSON de mapeo mencionado en el apartado anterior en la instrucción base:

Código 5.1: Carga del mapeo de propiedades e inyección en el prompt SPARQL

```
1 with open("mapeo_propiedades.json", "r", encoding="utf-8") as f:
2     mapeoProp = json.load(f)
3     textoMapeo = json.dumps(mapeoProp, indent=2)
4
5 def generar_sparql(consulta, id_entidad, textoMapeo,
6                   error_previo="", query_previo="", modelo='llama3'):
7     contexto_error = ""
8     if error_previo:
9         contexto_error = f"""
10         !ATENCION! Tu intento anterior fallo.
11         La consulta que generaste fue:
12         {query_previo}
13         Motivo del fallo: {error_previo}
14         Corrige el SPARQL para solucionar este problema.
15         """
16
17     prompt_sparql = f"""
18     Eres un traductor experto de lenguaje natural a SPARQL.
19     Esquema de propiedades disponibles:
20     {textoMapeo}
21     Objetivo: Genera una consulta SPARQL valida para '{consulta}'
22     {contexto_error}
23     REGLAS:
24     1. La entidad principal es: wd:{id_entidad}
25     2. Usa SOLO las propiedades del esquema de arriba.
26     ...
27     """
28     return consultar_llm(prompt_sparql, modelo)
```

5.4. Bucle de Reflexión y Autocorrección

La ejecución de la consulta SPARQL generada se encapsula en la función `ejecutar_sparql_local()` que lanza la consulta contra el endpoint de *Blazegraph* mediante *SPARQLWrapper*. Si *Blazegraph* devuelve un error de sintaxis, la función lo captura y lo retorna como cadena de texto. El sistema lo incorpora al siguiente *prompt* para que el agente pueda corregirse a sí mismo:

Código 5.2: Bucle de reflexión ante errores de Blazegraph

```
1 def ejecutar_sparql_local(consulta):
2     endpoint = "http://localhost:9999/blazegraph/namespace/kb/sparql"
```

```
3 prefijos = """
4 PREFIX wd: <http://www.wikidata.org/entity/>
5 PREFIX wdt: <http://www.wikidata.org/prop/direct/>
6 """
7 sparql = SPARQLWrapper(endpoint)
8 sparql.setQuery(prefijos + consulta)
9 sparql.setReturnFormat(JSON)
10 try:
11     resultados = sparql.query().convert()
12     return resultados["results"]["bindings"], None
13 except Exception as e:
14     error_str = str(e)
15     if "Connection refused" in error_str:
16         return [], "CONNECTION_ERROR"
17     return [], error_str # mensaje de error devuelto al agente
18
19 # En procesar_consulta(), el error se inyecta en el siguiente prompt:
20 resultados, error_db = ejecutar_sparql_local(query_limpia)
21 if not resultados and error_db:
22     error_previo = f"Error en Blazegraph: {error_db}"
23     # El agente reintentará con este contexto de error (max 3 intentos)
```

Este enfoque incrementó drásticamente la robustez del sistema, permitiéndole recuperarse de fallos comunes como puntos finales olvidados o prefijos `wdt` mal estructurados, sin mostrar errores en bruto al usuario final.

5.5. Interfaz Gráfica

La interfaz, construida con *Streamlit*, mantiene el estado de la conversación y proporciona una página intuitiva para el usuario. Destaca la inclusión de un panel desplegable de estado donde el usuario puede visualizar en tiempo real los pensamientos internos del agente: qué entidad extrae, qué consulta SPARQL ha generado y si ha necesitado autocorregirse. Esto dota al sistema de una robusta capa de explicabilidad (*Explainable AI*), diferenciándolo de los chatbots tipo "caja negra" convencionales.

6. Evaluación y Resultados

Para asegurar la viabilidad técnica del sistema y validar el cumplimiento de los objetivos establecidos en el Capítulo 3, se ha diseñado un entorno de pruebas automatizadas. Este capítulo expone la metodología de evaluación y discute los resultados cuantitativos obtenidos.

6.1. Entorno de Pruebas

Se elaboró un conjunto de datos (*dataset*) de validación en formato estructurado (`dataset_evaluacion.json`) que contiene preguntas en lenguaje natural junto con la entidad principal esperada, la propiedad del grafo requerida para resolverlas, y un listado de posibles respuestas correctas (`respuesta_esperada_contiene`) que actúan como *ground truth*.

El conjunto de pruebas se compone de 103 preguntas que abarcan diversas relaciones semánticas del dominio cinematográfico, evaluando tanto búsquedas directas (*single-hop*), en las que se preguntan propiedades específicas de una sola entidad, como consultas complejas de más de un salto (*multi-hop*) en las que se pregunta por propiedades de entidades relacionadas con la entidad inicial para relacionar múltiples nodos, por ejemplo, averiguar el lugar de nacimiento (`wdt:P19`) o país de ciudadanía (`wdt:P27`) del director o actor de una película concreta, abarcando todas las propiedades listadas en apartados anteriores.

El script `evaluacion.py` automatiza la ejecución secuencial de estas pruebas sin intervención humana. La evaluación se divide en tres fases: primero verifica la correcta extracción de la entidad inicial, luego valida la estructura y sintaxis de la consulta SPARQL generada, y finalmente comprueba que el dato real extraído de la base de datos local coincida con los valores estipulados en `respuesta_esperada_contiene`, que contarían como aciertos en la respuesta, calculando así la precisión global (*Accuracy*).

6.2. Resultados Obtenidos

Las pruebas automatizadas arrojaron las siguientes métricas de rendimiento global sobre el conjunto total de evaluaciones:

Modelo	Prec. Ent.	Prec. SPARQL	Prec. Respuesta	Prec. Final	T/Preg.	T. Total
llama3	97.1%	97.1%	97.1%	94.3%	9.51s	979.84s
llama3.2:1b	58.3%	21.4%	0.0%	0.0%	76.78s	7908.84s
mistral	82.5%	73.8%	94.6%	69.8%	10.57s	1088.79s
gemma2:9b	98.1%	97.1%	96.5%	93.7%	9.98s	1027.62s
gemma2:2b	85.4%	74.8%	73.7%	55.1%	20.28s	2088.85s
deepseek-r1:8b	91.3%	91.3%	95.9%	87.6%	35.59s	3665.37s

Tabla 6.1: Comparativa de rendimiento, precisión y latencia entre distintos LLMs locales evaluados.

En esta evaluación, también se hacía uso de la arquitectura iterativa diseñada (bucle de autocorrección) para comprobar su impacto real en la corrección de consultas SPARQL erróneas. Para ello, se evaluaron dos métricas principales:

- **Precisión Zero-Shot:** Porcentaje de consultas SPARQL generadas correctamente al primer intento, sin retroalimentación de la base de datos.
- **Precisión con Reflexión (Multi-Turn):** Porcentaje de acierto final permitiendo al agente iterar y corregir sus propios errores de sintaxis o semántica (hasta un máximo de 3 intentos).

La siguiente tabla muestra la diferencia de eficacia de los modelos al hacer uso de este bucle:

Modelo	Prec. Zero-Shot	Prec. Reflexión	Mejora Reflexión
llama3	92.2%	97.1%	+4.9%
llama3.2:1b	14.6%	21.4%	+6.8%
mistral	69.9%	73.8%	+3.9%
gemma2:9b	96.1%	97.1%	+1.0%
gemma2:2b	60.2%	74.8%	+14.6%
deepseek-r1:8b	89.3%	91.3%	+2.0%

Tabla 6.2: Impacto del Bucle de Reflexión sobre la precisión de generación SPARQL.

6.3. Análisis y Discusión de los Resultados

La inclusión de diferentes Modelos de Lenguaje (LLMs) locales ha permitido evaluar no solo la viabilidad cualitativa del sistema, sino también comparar su rendimiento bajo las mismas restricciones del sistema RAG. Los datos de la Tabla 6.1 revelan compromisos significativos según la arquitectura y el tamaño de cada modelo. A continuación, se detallan las conclusiones extraídas de cada métrica:

1. Rendimiento en Extracción de Entidades: El sistema demostró una altísima fiabilidad en esta primera fase, la cual es crítica para el éxito del resto de la consulta. Si el sistema no es capaz de aislar el sujeto de la oración (por ejemplo, extraer “Matrix” de la consulta “¿Quién dirigió Matrix?”), la búsqueda en la base de conocimientos fallará irremediablemente. La mayoría de los modelos evaluados superaron con creces el 80% de precisión al identificar este sujeto principal. Destacan especialmente **gemma2:9b** y **llama3**, alcanzando un excelente 98.1% y 97.1% respectivamente. Esto demuestra la gran capacidad de los modelos actuales de tamaño medio para comprender el contexto conversacional y aislar los términos clave independientemente de cómo el usuario formule la pregunta.

2. Generación de Código SPARQL: Traducir texto en lenguaje natural a la sintaxis estricta y tipada de SPARQL representa el mayor desafío técnico del proyecto. En este aspecto, tanto **llama3** como **gemma2:9b** se posicionaron como los líderes con un 97.1% de acierto final (tras el bucle de reflexión). Esto confirma la hipótesis de que los modelos con un tamaño en torno a los 8B-9B de parámetros poseen un razonamiento lógico y algorítmico superior para escribir código sin incurrir en alucinaciones (como inventar propiedades o relaciones inexistentes). Por el contrario, los modelos excesivamente pequeños sufren severamente en esta tarea; el caso más extremo es **llama3.2:1b**, cuyo rendimiento tras la reflexión cae al 21.4%. Esto indica que un modelo de apenas 1 billón de parámetros no retiene en su memoria interna la capacidad suficiente para dominar la ontología de Wikidata de forma fiable, ni siquiera contando con la ayuda del contexto inyectado.

3. Validación de Respuesta: Una vez que el agente ha generado una consulta SPARQL sintácticamente correcta y la ha ejecutado contra la base de datos, el paso final es validar si el resultado obtenido responde realmente a la intención original del usuario. Para esta evaluación, se consideró válida una respuesta si el modelo era capaz de generar un texto coherente y comprensible en lenguaje natural que reflejase fielmente la información extraída de la base de conocimientos.

El porcentaje representado en la tabla indica el índice de aciertos en la respuesta por parte del LLM tras ejecutar el SPARQL correctamente, por ejemplo, **mistral** tiene un 73.8% de acierto en la generación de código SPARQL, y de ese porcentaje, un 94.6% de las respuestas extraídas del SPARQL son correctas. Se escogió esta manera de evaluar la precisión de las respuestas ya que sin una consulta válida, la interpretación de los resultados es inviable.

Al observar esta métrica, se evidencia que los resultados son generalmente proporcionales a los obtenidos en la generación de código SPARQL, lo que sugiere que formular la consulta es la barrera principal. No obstante, destacan dos excepciones importantes: el modelo **llama3.2:1b** sufre una caída total en su eficacia, siendo incapaz de articular ninguna respuesta correcta (0.0%) pese a haber formulado bien algunas consultas (21.4%), mientras que **mistral** presenta el caso contrario, mejorando notablemente su rendimiento en esta fase final al alcanzar un 94.6% de acierto frente a su 73.8% en el paso anterior.

4. Análisis de Tiempos y Viabilidad: En un sistema interactivo orientado al usuario, la latencia (tiempo de respuesta) es un factor crítico para la experiencia final. Aquí sobresalen de manera espectacular **llama3** y **gemma2:9b**, resolviendo el flujo completo de pensamiento (desde la recepción de la pregunta hasta la ejecución final en la base de datos) en tiempos sumamente competitivos de 9.51s y 9.98s por pregunta de media, respectivamente. Esto representa un equilibrio idóneo entre precisión casi perfecta y velocidad.

En el lado opuesto, **deepseek-r1:8b** (35.59s por pregunta) y **llama3.2:1b** (76.78s), resultan demasiado lentos para un chat en tiempo real. Es importante destacar un fenómeno particular: el altísimo tiempo registrado por **llama3.2:1b** no se debe a que

la inferencia del modelo sea lenta en sí misma (de hecho, es muy ligero), sino a que su baja precisión inicial provoca que el sistema active constantemente el “Bucle de Reflexión”. Al generar código SPARQL erróneo con frecuencia, el sistema de corrección intercepta el error y obliga al modelo a reescribir la consulta repetidas veces. Al fallar recurrentemente en corregirse a sí mismo, agota los intentos máximos permitidos, lo que dispara el tiempo total de ejecución y la media por pregunta.

5. Eficacia del Bucle de Autocorrección: En esta tabla se comparan los resultados de la primera consulta SPARQL generada por los modelos con la consulta definitiva y corregida. Es importante puntualizar la diferencia de precisión de los modelos con gran capacidad como *Llama 3* o *Deepseek-r1:8b*, de 8B parámetros cada uno, y modelos más ligeros como *Llama3.2:1b* o *Gemma2:2b*, de 1B y 2B parámetros respectivamente.

Como se observa en los resultados, el modelo de mayor tamaño (*Gemma2:9b*) posee una capacidad de razonamiento *Zero-Shot* muy sólida (96.1%), requiriendo menos asistencia del bucle para alcanzar su sobresaliente 97.1% final.

Sin embargo, el verdadero impacto de la arquitectura iterativa es evidente al analizar el modelo ligero (*Gemma 2 2B*). Este modelo presenta inicialmente una reducida tasa de acierto (60.2% en *Zero-Shot*) debido a frecuentes fallos de sintaxis o lógica en su primer intento, pero gracias al mecanismo de autocorrección desarrollado, el agente es capaz de leer el *traceback* del error devuelto por la base de datos y reescribir la consulta, logrando una mejora del +14.6% en su rendimiento final, superando así finalmente a *mistral*, modelo de 7B parámetros.

Esto demuestra empíricamente que la arquitectura iterativa compensa las limitaciones de los modelos más pequeños, permitiéndoles recuperar consultas que originalmente habrían fallado y aumentando significativamente la resiliencia general del sistema, además de mejorar también los modelos más capaces para aumentar su precisión final.

6.4. Conclusiones de la Evaluación

En conclusión, la elección del motor LLM para un sistema de estas características requiere un claro y delicado compromiso entre latencia y precisión. Para un balance óptimo en un entorno de producción local, tanto *llama3* como *gemma2:9b* representan las opciones más robustas. Ambos ofrecen un rendimiento muy parejo, superando el 93% de precisión en la respuesta final y manteniendo tiempos de inferencia en torno a los 10 segundos por pregunta, logrando así dotar al Grafo de Conocimiento de una interfaz verdaderamente inteligente y fiable que oculta por completo la complejidad técnica de la Web Semántica al usuario.

7. Conclusiones y Trabajo Futuro

Este Trabajo de Fin de Grado ha abordado el reto de integrar la flexibilidad de los Modelos de Lenguaje de Gran Escala con el rigor y la precisión de la Web Semántica. El sistema desarrollado ha demostrado ser una solución técnica viable para consultar información estructurada compleja mediante lenguaje natural, mitigando activamente el problema de las alucinaciones en los LLMs.

7.1. Consecución de Objetivos

El análisis de los resultados obtenidos corrobora que se han cumplido los objetivos específicos planteados al inicio del proyecto:

- **Precisión en la Generación de SPARQL:** Se ha logrado una sobresaliente precisión en la traducción autónoma de texto libre a código SPARQL en el dominio cinematográfico, alcanzando hasta un 97.1% de acierto final con modelos como `llama3` o `gemma2:9b`. Esto ha sido posible gracias al exitoso diseño e implementación de un bucle iterativo de autocorrección (reflexión) que permite al agente enmendar sus propios errores, superando con creces los índices de éxito que hubieran tenido en un intento único (*Zero-Shot*).
- **Manejo Avanzado de Datos RDF:** Se ha construido un Grafo de Conocimiento local optimizado, extrayendo datos reales de Wikidata y volcándolos en formato *N-Triples*. El uso de *Blazegraph* como motor SPARQL local ha permitido mantener tiempos de respuesta ágiles, eliminando las elevadas latencias y la dependencia de red que supondría conectarse a servicios externos en cada consulta.
- **Control y Restricción del LLM:** Se ha validado la eficacia de un marco de inyección dinámica de propiedades semánticas en el *prompt*. Esto ha servido para limitar estrictamente la visión del modelo de lenguaje, forzándolo a utilizar únicamente las relaciones permitidas y mitigando así de forma contundente las alucinaciones inherentes de los LLMs.
- **Evaluación Cuantitativa:** Se ha implementado un banco de pruebas riguroso (`evaluacion.py`) sin intervención humana. Esta batería ha permitido medir objetivamente y comparar entre varios LLMs el desempeño del sistema completo: desde la extracción inicial de la entidad, pasando por la robustez del código SPARQL, hasta la precisión y latencia de la respuesta final devuelta al usuario.

7.2. Líneas de Trabajo Futuro

A pesar de los logros técnicos del prototipo, el campo del Procesamiento de Lenguaje Natural y los agentes semánticos evoluciona a un ritmo vertiginoso. Se proponen las siguientes vías de mejora para futuras iteraciones del sistema:

- **Mejora en la desambiguación de entidades:** El 20% de los fallos de extracción derivó de problemas idiomáticos o nombres ambiguos ("Origen" vs "Inception"). Implementar algoritmos de similitud semántica mediante búsqueda vectorial (*embeddings*) podría mitigar la dependencia de la API restrictiva de Wikidata.
- **Ampliación controlada de la ontología:** Integrar más propiedades en el diccionario base (por ejemplo, "recaudación en taquilla", "premios obtenidos" o "banda sonora") requerirá técnicas avanzadas para evitar que el incremento del espacio de búsqueda confunda al LLM y degrade su precisión.
- **Soporte robusto para consultas *Multi-Hop* (Multi-salto):** Integrar ejemplos *Few-Shot* dinámicos en el *prompt* para que el agente pueda resolver eficientemente preguntas que implican recorrer varios nodos del grafo secuencialmente (ej. "¿Qué directores han trabajado con actores ganadores de un Oscar?").
- **Interfaz de usuario:** Dado que Streamlit presenta ciertas limitaciones a la hora de personalizar completamente la interfaz gráfica y escalar a miles de usuarios concurrentes, como trabajo futuro se propone la migración del Frontend a un framework web reactivo moderno (como Vue.js o Next.js). Esto implicaría desacoplar completamente la lógica de negocio creando una API RESTful en FastAPI, lo que permitiría construir una experiencia de usuario (UX) más dinámica y personalizada.

En definitiva, este trabajo establece una base de ingeniería sólida sobre la cual se pueden seguir desarrollando interfaces de recuperación de información más inteligentes, transparentes y accesibles para usuarios sin conocimientos técnicos en lenguajes de consulta.

Bibliografía

- [1] W3C. Rdf 1.1 concepts and abstract syntax, 2014. URL <https://www.w3.org/TR/rdf11-concepts/>. Accedido: 2024.
- [2] W3C. Sparql 1.1 query language, 2013. URL <https://www.w3.org/TR/sparql11-query/>. Accedido: 2024.
- [3] Denny Vrandečić and Markus Krötzsch. Wikidata: a free collaborative knowledge-base. *Communications of the ACM*, 57(10):78–85, 2014.
- [4] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [5] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.
- [6] LangChain, Inc. Langchain documentation, 2024. URL <https://python.langchain.com/>. Accedido: 2024.
- [7] DataCamp. Knowledge graphs and llms, 2024. URL <https://www.datacamp.com/es/blog/knowledge-graphs-and-llms>. Accedido: 2024.

Lista de Acrónimos y Abreviaturas

API	Application Programming Interface.
JSON	JavaScript Object Notation.
KG	Knowledge Graph.
LLM	Large Language Model.
NLP	Natural Language Processing.
NT	N-Triples.
PLN	Procesamiento de Lenguaje Natural.
RAG	Retrieval-Augmented Generation.
RDF	Resource Description Framework.
SPARQL	Simple Protocol and RDF Query Language.
UI	User Interface.

A. Manual de Instalación y Despliegue

En este anexo se detallan los pasos necesarios para instalar, configurar y desplegar el sistema *MMT Chat* en un entorno local. El sistema está compuesto por tres pilares fundamentales: la base de datos orientada a grafos (Blazegraph), el gestor de modelos de lenguaje (Ollama) y la interfaz de usuario desarrollada en Python (Streamlit).

A.1. Requisitos Previos

Para garantizar el correcto funcionamiento del sistema, el entorno de despliegue debe contar con los siguientes requisitos mínimos:

- **Docker** instalado y en ejecución en el sistema (necesario para desplegar Blazegraph).
- **Python** versión 3.9 o superior.
- **Ollama** instalado en el sistema operativo para la ejecución local de los modelos de lenguaje (LLMs).

A.2. Despliegue de Blazegraph

Blazegraph actúa como el motor de persistencia para el grafo de conocimiento subyacente. En este proyecto se opta por su despliegue mediante contenedores para simplificar la configuración. Para su puesta en marcha:

1. Asegurarse de que el demonio de Docker está activo.
2. Abrir una terminal y ejecutar el siguiente comando para descargar la imagen oficial y levantar el contenedor, mapeando el puerto 9999 y asignando un nombre al contenedor:

```
docker run -p 9999:8080 -d --name mmt-blazegraph lyrrasis/blazegraph↵  
↵ :2.1.5
```

Una vez iniciado el contenedor, la interfaz de administración y el *endpoint* SPARQL estarán disponibles por defecto en <http://127.0.0.1:9999/blazegraph/>.

A.3. Configuración de Ollama y Modelos Locales

Para habilitar la generación de respuestas y el funcionamiento del agente reflexivo, es necesario descargar los modelos de lenguaje locales correspondientes a través de Ollama.

1. Asegurarse de que el servicio de Ollama se encuentra en ejecución en el sistema (por defecto se expone en `http://127.0.0.1:11434`).
2. Abrir una terminal y ejecutar los comandos para descargar los modelos que se vayan a utilizar (por ejemplo, Llama 3 o similares). Para el modelo principal:

```
1 ollama pull llama3
```

(Nota: Se deberá hacer un `pull` adicional por cada modelo específico con el que se desee evaluar el sistema).

A.4. Configuración del Entorno Python

El núcleo lógico del chatbot, la orquestación del grafo y la interfaz de usuario están desarrollados en Python. Se recomienda encarecidamente el uso de un entorno virtual para mantener aisladas las dependencias del proyecto.

1. Crear y activar un entorno virtual dentro de la carpeta del proyecto:

```
1 # Creación del entorno virtual
2 python -m venv env
3
4 # Activación en sistemas Windows:
5 env\Scripts\activate
6
7 # Activación en sistemas Linux/macOS:
8 source env/bin/activate
```

2. Con el entorno activado, instalar las dependencias requeridas leyendo el archivo `requirements.txt`:

```
1 pip install -r requirements.txt
```

A.5. Ejecución de la Interfaz (Streamlit)

Una vez que Blazegraph y Ollama están en ejecución, y las dependencias de Python instaladas, se puede lanzar la interfaz gráfica:

1. En la terminal con el entorno virtual activado, navegar hasta el directorio raíz del código fuente.
-

2. Ejecutar el comando para iniciar la aplicación Streamlit (por defecto, el punto de entrada es `interfaz.py`, adaptarlo en caso de utilizar otro nombre):

```
streamlit run interfaz.py
```

Tras lanzar este comando, la consola mostrará las URLs locales de acceso, y automáticamente se abrirá una nueva pestaña en el navegador web predeterminado (típicamente en `http://localhost:8501`), permitiendo al usuario comenzar a interactuar de forma inmediata con MMT Chat.